

TP3 (JavaScript Version)

INF8808 : Data Visualisation

Department of computer and software engineering



POLYTECHNIQUE MONTRÉAL

Author : Olivia Gélinas

Teacher's Assistant : Hellen Vasques

Objectives

The goal of this lab is to create an interactive heatmap using open data in CSV format.

Before beginning, we recommend you have completed the following readings and practice exercises :

Readings : Chapter 9 (p.149 - p.158), 10, and 11 of Scott Murray's book

- Exercices :**
- Chapter 6 – 1, 2, 3
 - Chapter 8 – 1, 2

Introduction

A heatmap is a type of data visualization which shows values depicted as colors in two dimensions. The hue or intensity of the color represents the magnitude of the observation for its corresponding values in 2 dimensions.

In this lab, you will implement an interactive heatmap using data representing tree plantations over time in Montreal neighborhoods. The data was extracted from the City of Montreal's open data portal [1]. The dataset was then modified into a version to be used in this lab. It contains data representing the date and location that public trees were planted in Montreal.

Description

In this lab, you will have to complete the JavaScript code using D3 in order to display a heatmap representing how many trees were planted in each Montreal neighborhood each year between 2010 and 2020. To make the chart interactive, you will also implement the code to display a cell's value when it is hovered. Additionally,

there will be a legend indicating the color corresponding to the amount of trees planted throughout the years in Montreal's neighborhoods.

The following subsections present the different parts that you will have to complete for this lab. While you code, we recommend completing the data processing first, followed by the implementation of the bar chart itself. The next two parts, the legend and the hover information are independent of each other.

File Structure

To complete this work, you will need to fill the various **TODO** sections in the files from the archive provided for the lab. The comments in the code explain in more detail the steps to take.

In this lab, we provide you with an archive containing 7 JavaScript files used to accomplish the desired visualization :

- **index.js** : This file represents the entry point to the code and orchestrates the various steps needed to realise the visualization. It does not need to be modified.
- **scripts/helper.js** : This file contains some basic functions needed to display the visualization. It does not need to be modified.
- **scripts/hover.js**
- **scripts/legend.js**
- **scripts/preprocess.js**
- **scripts/util.js** : This file contains a small utility function which will be passed as a parameter when it may be useful to you. This file does not need to be modified.
- **scripts/viz.js**

Dataset

The dataset is located in the **src/assets/data/** directory in the archive provided for the lab. The dataset contains the following columns :

- **Arrond** : The ID of the neighborhood where the tree was planted.
- **Arrond_Nom** : The name of the neighborhood where the tree was planted.
- **Date_Plantation** : The date the tree was planted.
- **Longitude** : The longitude of the tree.
- **Latitude** : The latitude of the tree.

Data preprocessing

To begin, you will have to preprocess the data we provide you. The data contained in the CSV file is raw, so it is necessary to reorganize certain parts of it so they can be properly used by the D3 library. To do so, you need to complete the file **scripts/preprocessing.js**.

More precisely, you will have to complete these steps :

1. Read from the data and extract a list of neighborhood names (function `getNeighborhoodNames`)
2. Filter the data by year (function `filterYears`). Though we have data from trees planted starting in the early 20th century, we want to focus on tree plantations between 2010 and 2020 for this visualization.
3. Summarize the data to get the total count of trees planted per year per neighborhood (function `summarizeYearlyCounts`)
4. Fill any missing counts with the value 0 (function `fillMissingData`)

The Snippet 1 below shows an extract of the data table which should result from these steps. The snippet shows one of the objects which should be contained in the table.

```
{
  "Arrond_Nom": "Ville-Marie",
  "Plantation_Year": 2017,
  "Counts": 888
}
```

Snippet 1 : Resulting object for neighborhood Ville-Marie in 2007

Heatmap

For this second part, you will have to implement the main part of the data visualization. First, you will have to update the color scale used to color the rectangles in the visualisation. Second, you will use the data to append the rectangles to the heatmap. Third, you will update the X and Y scales according to the data and the visualization size and draw the axes accordingly. Fourth, you will update the position, size and color of the rectangles you previously appended, resulting in the completed heatmap. To attain this objective, you need to complete the file `scripts/viz.js`.

These are the steps you will have to accomplish for this part :

1. Update the color scale's domain to reflect the data (function `setColorScaleDomain`)
2. Append the SVG structure and bind the data (function `appendRects`). This structure will be used to draw the heatmap in the next steps.
3. Update the scales used to determine the positions in x and in y to reflect the data and the heatmap size (functions `updateXScale` and `updateYScale`). Make sure the neighborhood names are sorted alphabetically.
4. Draw the x and y axis respectively at the top and right of the graphic, with the ticks on the y axis rotated 30 degrees (functions `drawXAxis`, `drawYAxis`, and `rotateYTicks`)
5. Update the visual appearance of the rectangles, positioning and sizing them according to the x and y scales (function `updateRects`). Their color should be chosen according to the color scale.

Figure 1 illustrates the expected appearance of the heatmap following these steps (the legend will be added in the next step).

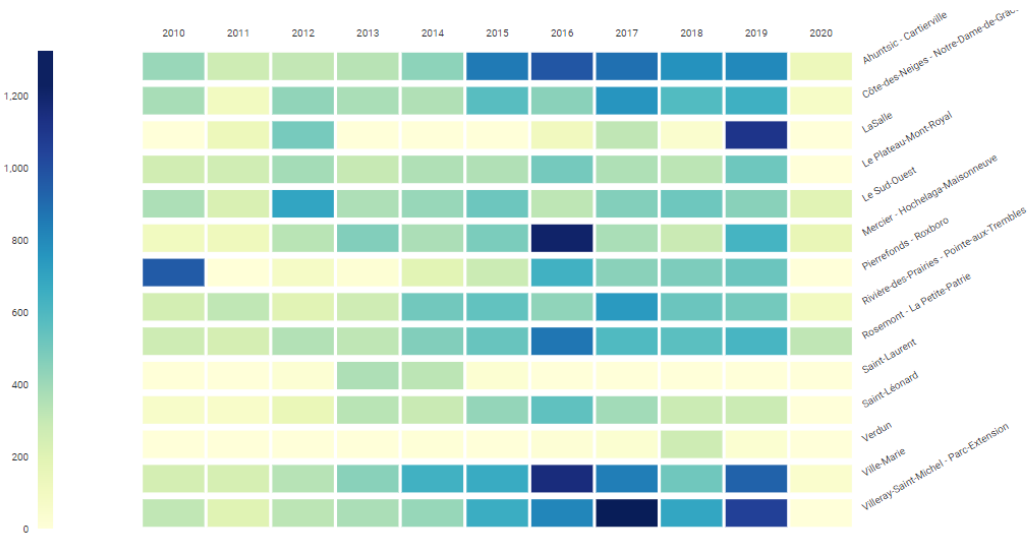


Figure 1 : The heatmap

Legend

For this third part, you will generate a vertical legend to the left of the heatmap. The legend will indicate the number of trees planted each color corresponds to in the continuous color scale. Some of the code for this part has already been implemented.

The provided code initializes the gradient used as the legend's fill, initializes the rectangle to represent the legend's color bar, and initializes the group for the legend's axis. Your task will consist in drawing the legend itself using these elements. To complete this part, you need to complete the code in `scripts/legend.js`.

The step to follow to accomplish this part is as follows :

- 1. Draw the color bar for the legend with its corresponding axis (function `draw`)

The result of the step is illustrated in Figure 2. Pay close attention to the visual appearance of each element.

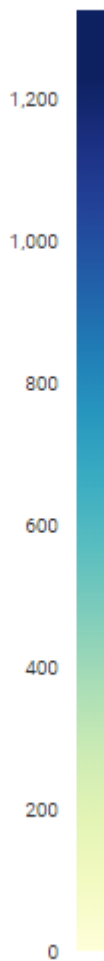


Figure 2 : The legend

Hover information

For this fourth part, you will implement the hover behavior for the heatmap. The goal of the feature is to help make the heatmap easier to read. When a rectangle is hovered, we wish for its corresponding tree count value to be displayed in its center. The ticks on the x and y axes which correspond to the hovered rectangle should also display their text in bold while the rectangle remains hovered. The code to complete to accomplish this part is located in the file `scripts/hover.js`.

More specifically, the steps you will need to complete for this part are :

1. Call the appropriate functions when a rectangle gets hovered by the cursor (function `setRectHandler`)
2. When a rectangle is hovered, display its number of trees in its center (in black, or in white if the value is 1000 or more). Make sure the rectangle are displayed at 75% opacity while hovered and 100% opacity otherwise (functions `rectSelected` and `rectUnselected`)
3. While a rectangle is hovered, make its corresponding ticks on the x and y axes bold (functions `selectTicks` and `unselectTicks`)

The Figures 3 and 4 that follow illustrate the expected behavior when a rectangle is hovered by the cursor.

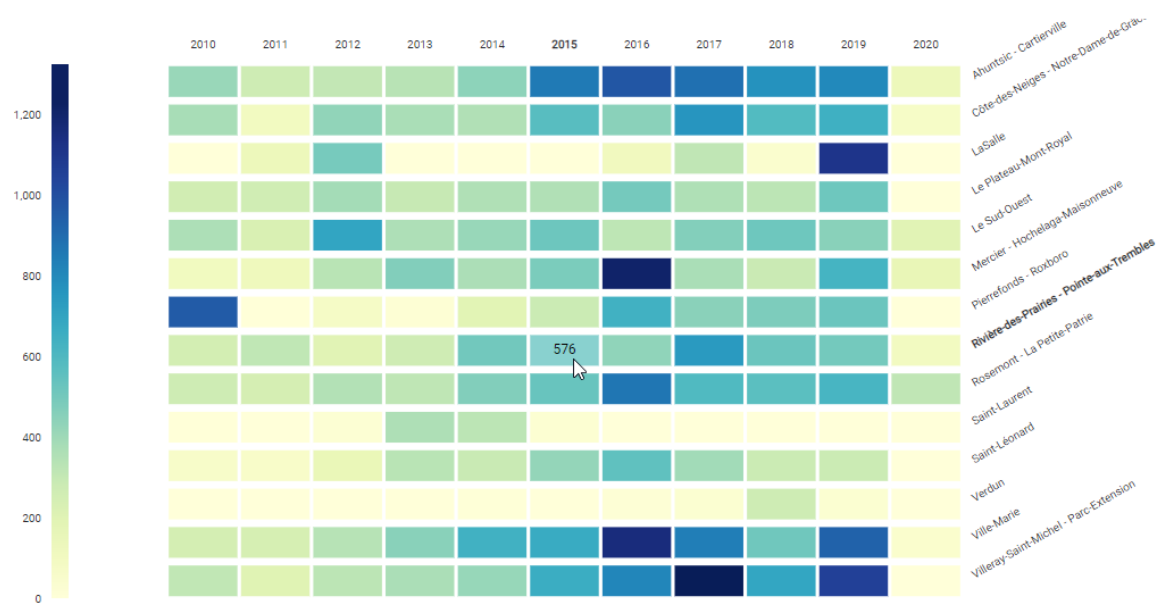


Figure 3 : The heatmap with the neighborhood Rivière-des-Prairies - Pointe-aux-Trembles and the year 2015 hovered

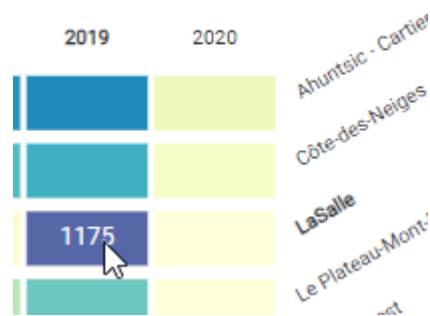


Figure 4 : The heatmap with the neighborhood LaSalle and the year 2019 hovered

Submission

The instructions for the submission are :

- 1. You must place your project code in a compressed ZIP file named [StudentID1_StudentID2_StudentID3.zip]
- 2. The lab must be submitted on the JS box on Moodle

Evaluation

Overall, your work will be evaluated according to the following grid. Each section will be evaluated on correctness and quality of the work.

Requirement	Points
Data preprocessing	4
Heatmap	8

Requirement	Points
Legend	3
Hover information	4
Overall quality and clarity of the submission	1
Total	20

References

[1] Service des grands parcs, du Mont-Royal et des sports, "Arbres publics sur le territoire de la Ville ," Montréal : Portail de données ouvertes. Available: <http://donnees.ville.montreal.qc.ca/dataset/arbres> [Accessed 01 09 2020].